Kotlin Coroutines, Retrofit, Dependency Injection, Architecture

Android Lecture 4

Simona Kurňavová

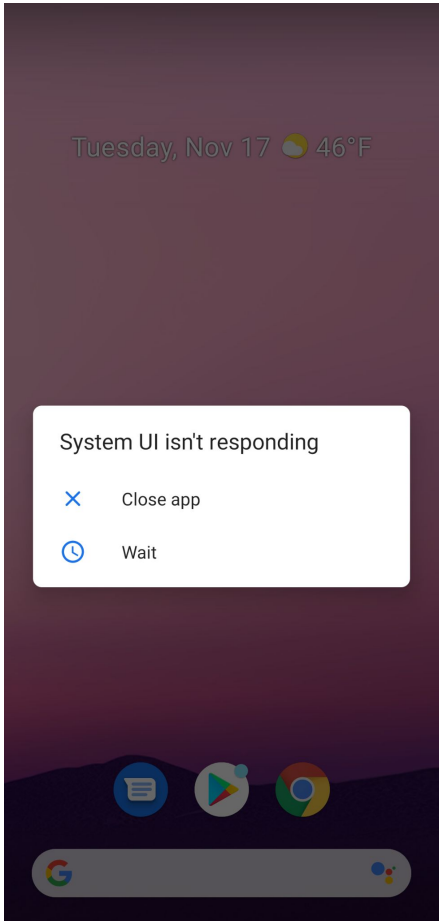
Today’s Agenda

- Background processing & Kotlin Coroutines
- Retrofit
- Dependency injection
- Persistence
- Architecture
- Q&A

Background processing

Motivation

Keep your application responsive



Background processing

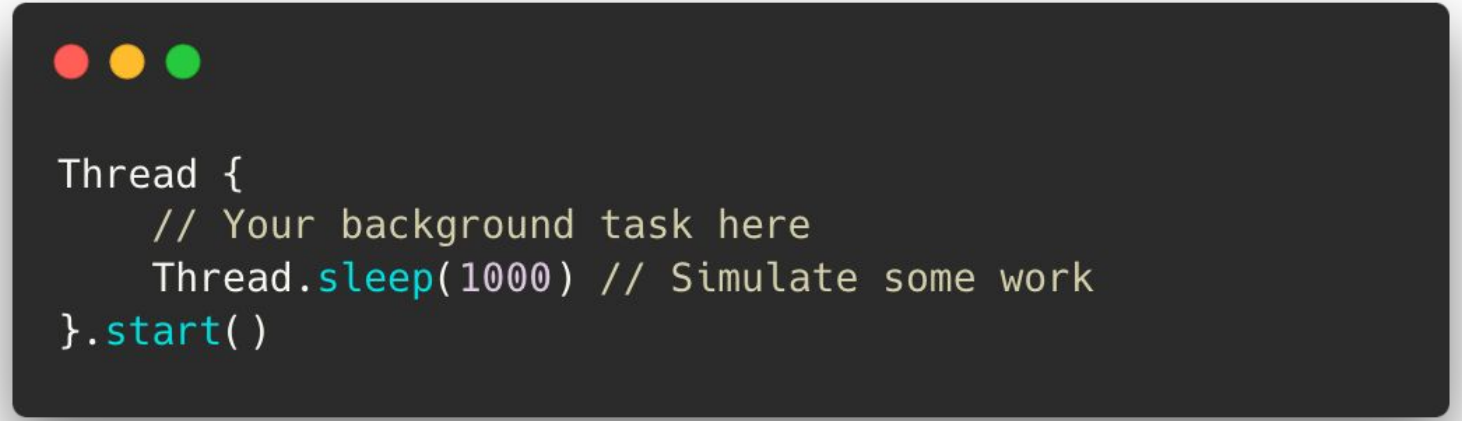
- Main thread = UI thread
- Avoid long running operations on Main/UI thread
  - Files, database operations, network, …
- Most components run on Main thread by default
- 5 second to ANR (10s BroadcastReceiver)
- **Never block UI thread!**

Background processing: Options

- Threads
- Handler
- RxJava
- **Kotlin coroutines**
- ~~AsyncTask~~ - Deprecation in Android 11 (API-30)
- ~~Loader~~ Deprecation in Android 9 (API-28)

Thread

- *java.lang.Thread*
- Standard Java thread
- Simple way how to offload work to the background
- UI can’t be updated from background

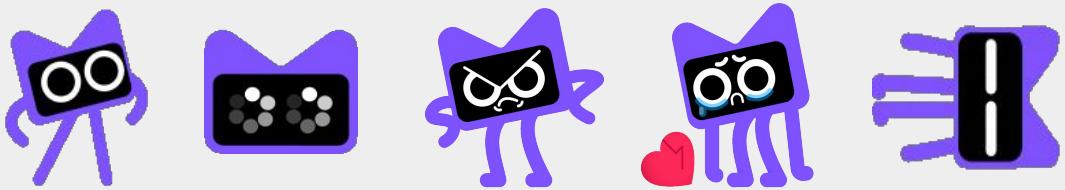


```
Thread {
    // Your background task here
    Thread.sleep(1000) // Simulate some work
}.start()
```

“Some people, when confronted with a problem, think, “I know, I’ll use threads,” and then they have two problems.”

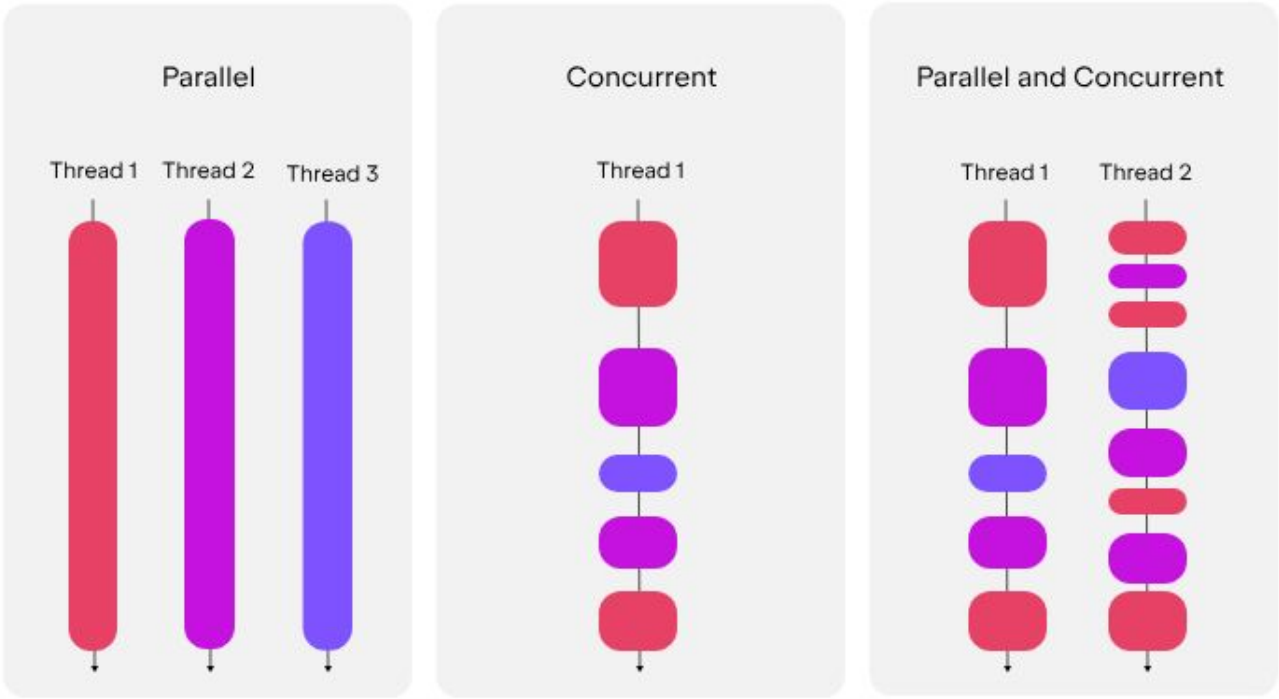
Jamie Zawinski

Source



Kotlin Coroutines

Parallelism vs. Concurrency



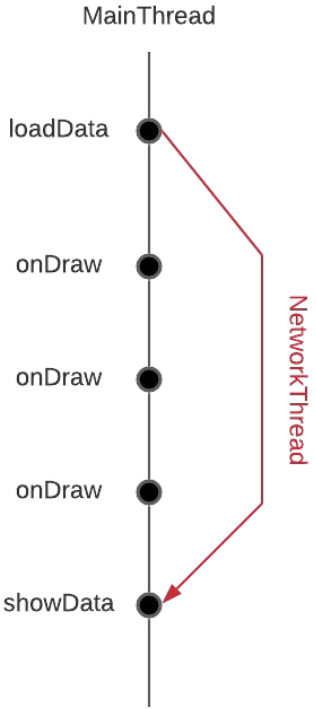
Kotlin Coroutines

- Added in Kotlin 1.3
- Coroutine = Lightweight thread. An instance of suspendable computation.
- Creating a new coroutine is cheap and efficient (unlike creation of thread)
- Jetpack integration: many libraries provide support of coroutines: custom scopes, extension functions…

Kotlin Coroutines

Suspend functions:

- **Function which is able to suspend it's execution without blocking thread**
- **Suspend/resume**
  - Internally pass Continuation object as callback
  - Uses finite state machine under the hood
- **Can be called only from other suspend function or in Coroutine**
- **suspend does not mean to run function on background!**



Kotlin Coroutines

```kotlin
suspend fun fetchDataFromNetwork(): String {
    // Simulate a delay (like network latency)
    delay(2000)  // delay for 2 seconds

    // Return some data after the delay
    return "Network data fetched successfully"
}
```

Kotlin Coroutines

Coroutine Builders:

- Builders create new Coroutine
- Accepts lambda suspend function that defines this Coroutine
- Types:
  - launch: Launch and forget. Returns **Job** object that can be used to cancel the Coroutine.
  - async: Launch and provide result. Returns **Deferred** object that will contain the result after calling *.await()* method.
  - runBlocking: Creates Coroutine that blocks the current thread - it will wait for its finish. **Do not use in production code!**

Kotlin Coroutines

Coroutine scope:

- Every Coroutine need to run in some scope
- Scope keeps track of every Coroutine inside it
- If scope cancels, every Coroutine in this scope cancels
- Types:
  - **GlobalScope**: Lifetime scope of application, not recommended to use in most cases
  - **Pre-defined Android scope**: *viewModelScope* in ViewModel, *lifecycleScope* in Activity
  - **Custom scope**

Kotlin Coroutines

Launching a new Coroutine with Coroutine builder launch in ViewModel Coroutine scope.

```
class UserViewModel: ViewModel() {

    fun fetchData(username: String) {
        viewModelScope.launch {
            state.value = LoadInProgress
            fetchDataSuspend(username)
        }
    }
}
```

Kotlin Coroutines

Dispatchers:

- All coroutines run in Dispatcher
- Dispatcher is responsible for managing the execution of coroutines on a thread / set of threads
- Coroutines can suspend themselves
- Knows how to resume suspended coroutines
- Part of coroutine context

Kotlin Coroutines

Dispatcher types:

- Dispatchers.Main: Main/UI thread. Interaction with UI, very light work. One thread only per Android application.
- Dispatchers.IO: Optimised for input/output operations (networking, database access, …). Multiple available threads in thread pool.
- Dispatchers.Default: Optimised for CPU intensive operations (complex algorithms, data filtering, calculations, …). Thread pool is defined by the number of CPU cores.

Note: When not specifying Dispatcher when running coroutine, Main dispatcher is used by default in most cases!

Kotlin Coroutines

Creating new Coroutine scope using Dispatchers.IO, launching new coroutine and then switching Dispatcher to Dispatchers.Default.

```
fun fetchData() {
    // Launch a new coroutine on the IO dispatcher
    CoroutineScope(Dispatchers.IO).launch {
        val result = performNetworkRequest()

        // Switch to Default dispatcher
        withContext(Dispatchers.Default) {
            performSomeCalculations(result) // CPU intensive suspend functions
        }
    }]
}

suspend fun performNetworkRequest() {
  // Call network to fetch some data
}
```

Kotlin Coroutines

Suspend functions should be **main-safe**:

A suspend function can be safely invoked on the main thread without causing UI freezes or performance issues

- Do not use Main/UI thread for long running operations
- Use non-blocking operations (for example: do not use Thread.sleep())
- Check main-safety for third party libraries
- Use correct dispatchers for operations

HTTP Clients

Retrofit

- Documentation
- Developed by Square
- Type-safe HTTP client for Android and Java
- Allows synchronous and asynchronous requests
- Simplifies definition of API endpoints and making requests

Retrofit: Create API

```
interface ApiService {
    // GET request
    @GET("users/{id}")
    suspend fun getUser(@Path("id") userId: String): Response<User>

    // POST request
    @POST("users")
    suspend fun createUser(@Body user: User): Response<User>
}
```

Retrofit: Create API

- @Path: variable that is a part of URL path
  - id in GET /users/{id}
- @Query: variable that is used as query
  - age and sort in GET /users?age=25&sort=desc
- @Body: body of request
  - POST /users { "name": "John Doe", "email": "johndoe@example.com" }

Retrofit: Create Retrofit instance and make a call

```
val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/") // Base URL of API
    .addConverterFactory(GsonConverterFactory.create()) // Converter for JSON -> Object
    .build()

val apiService = retrofit.create(ApiService::class.java)

// Make a network call
coroutineScope.launch(Dispatchers.IO) {
    val response = apiService.getUser("123")
    if (response.isSuccessful) {
        val user = response.body() // Handle the response data
        println("User name: ${user?.name}")
    } else {
        // Handle error
    }
}
```

Ktor

- Documentation
- Alternative to Retrofit
- Multiplatform support
- Asynchronous by default
- Modular features: authentication, logging, etc.

Dependency Injection

Dependency Injection

Dependency Injection (**DI)** is a way to provide an object with all the resources or services it needs (dependencies) rather than having the object create those resources on its own.

Why:

- Makes creation of objects easier
- Centralizes creation of dependencies
- Easy to swap dependencies (for example for unit tests)

Dependency Injection for Android

- Dagger/Hilt: Very robust and quite complex
- Koin: Simple and easy to setup
- KodeIn

Koin

- Documentation
- Koin module = defines available dependencies and how to create them
  - factoryOf / factory {} - creates new instance of class each time it is requested
  - singleOf / single {} - creates singleton instance of class that lives across whole life of the app
  - viewmodelOf / viewmodel {} - special constructor for Android ViewModels
- Access to defined dependencies:
  - Method get() eagerly
  - Using by inject() for a class property (lazy evaluation)

Persistence

About Persistence

- Refers to the ability to save data in a way that it remains available even after the application or device is restarted or session changed
- **What to store**: cache, preferences (theme, notifications, language, ...), important user information (account, saved data, ...)
- **How to store:** database, files, sharedPreferences/data store, cloud, ...
- **What to take into account**: type and amount of data, security, for how to long to store them, backup on cloud, etc.

Persistence: Options

- Database: **Room,** SQLiteDatabase (low level DB)
- Preferences: **DataStore**, ~~SharedPreferences~~ (deprecated)
- Files
- Backend

Data store

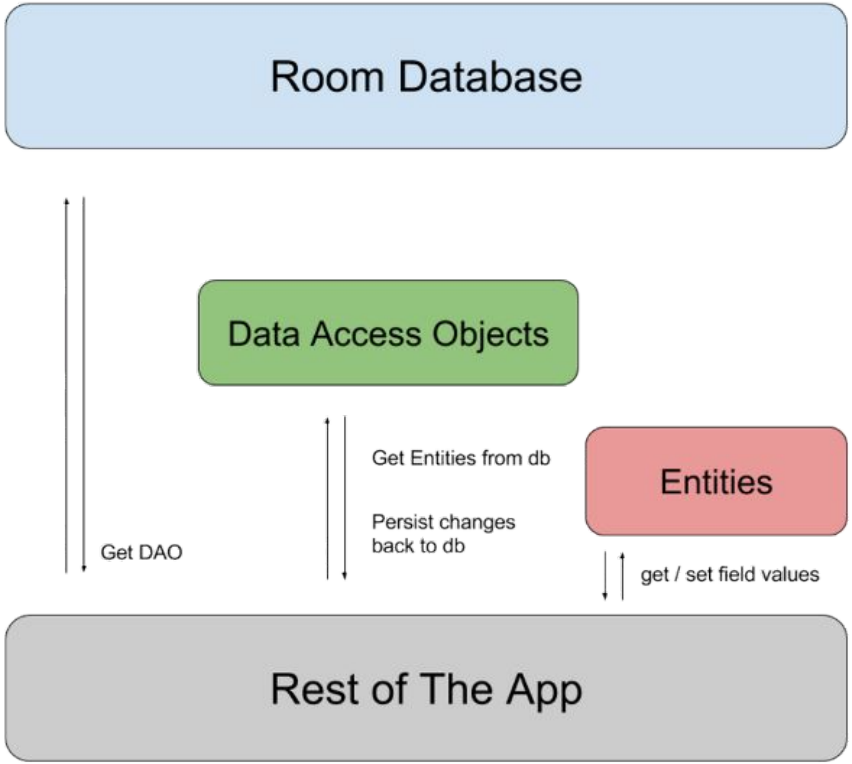
- Documentation
- Key-value storage for small data
- Asynchronous
- Should not be used instead of standard database
- Replaces SharedPreferences (Deprecated)
- Two implementations:
  - **Preferences data store:** storing key-value pairs without defining a schema.
  - **Proto data store**: stores structured data using Protocol Buffers, a schema-driven, type-safe format that’s useful for more complex data structures.

Data store: Preferences DataStore

```kotlin
// Use the property delegate created by preferencesDataStore to create an instance of DataStore.
// Call it once at the top level of your kotlin file. This makes it easier to keep DataStore as a singleton.
val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "settings")

// Define keys for each preference item
val USER_NAME_KEY = stringPreferencesKey("user_name")
val USER_AGE_KEY = intPreferencesKey("user_age")

fun saveUserPreferences(context: Context, userName: String, userAge: Int) {
    CoroutineScope(Dispatchers.IO).launch {
        context.dataStore.edit { preferences ->
            preferences[USER_NAME_KEY] = userName  // Store a string
            preferences[USER_AGE_KEY] = userAge    // Store an integer
        }
    }
}

fun getUserPreferences(context: Context): Flow<String> =
    context.dataStore.data
        .map { preferences ->
            preferences[USER_NAME_KEY] ?: "Default Name"
        }
```

Room

- Documentation
- Abstraction over SQLite
- Compile time validation of SQL queries
- Full integration with other Architecture components (LiveData, LifecycleObserver)
- Multiplatform support
- Often used with Repository pattern



Room: Entities

- Represents a table

```
@Entity
data class Car(
    @PrimaryKey val id: Int,
    @ColumnInfo(name = "manufacturer") val manufacturer: String?,
    @ColumnInfo(name = "model") val model: String?,
    @ColumnInfo(name = "nubmer_of_wheels") val numberOfWheels: String?
)
```

Room: DAO

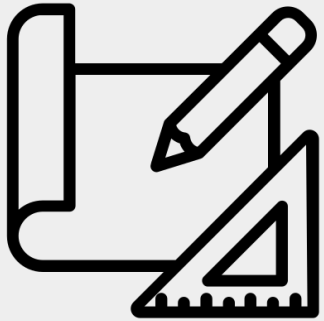
- Defines operations on top of entities

```
@Dao
interface CarDao {
    @Query("SELECT * FROM car")
    fun getAll(): List<Car>

    @Query("SELECT * FROM car WHERE id IN (:carIds)")
    fun loadAllByIds(carIds: IntArray): List<Car>

    @Query("SELECT * FROM car WHERE manufacturer LIKE :manufacturer AND " +
           "model LIKE :model LIMIT 1")
    fun findByModel(manufacturer: String, model: String): Car

    @Insert
    fun insertAll(vararg cars: Car)

    @Delete
    fun delete(car: Car)
}
```

Room: Database

- Defines database

```
@Database(entities = arrayOf(Car::class), version = 1)
abstract class CarDatabase : RoomDatabase() {
    abstract fun carDao(): CarDao
}

val db = Room.databaseBuilder(
            applicationContext,
            CarDatabase::class.java, "car-database"
        )
.addMigrations(...)
.build()
```



Clean Architecture

Software Architecture

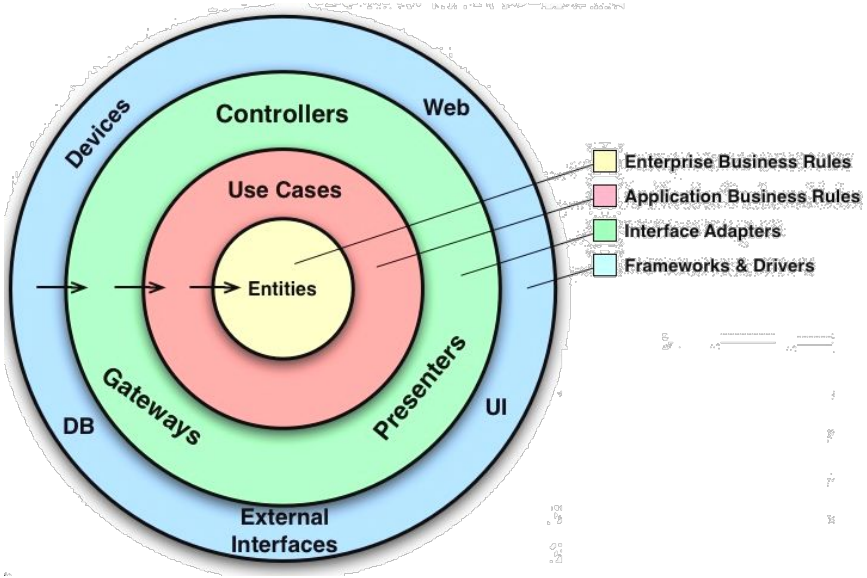
High level structure of software, defines building blocks and their interactions.

Benefits:

- Readability
- Scalability
- Reusability
- Maintainability

Clean Architecture

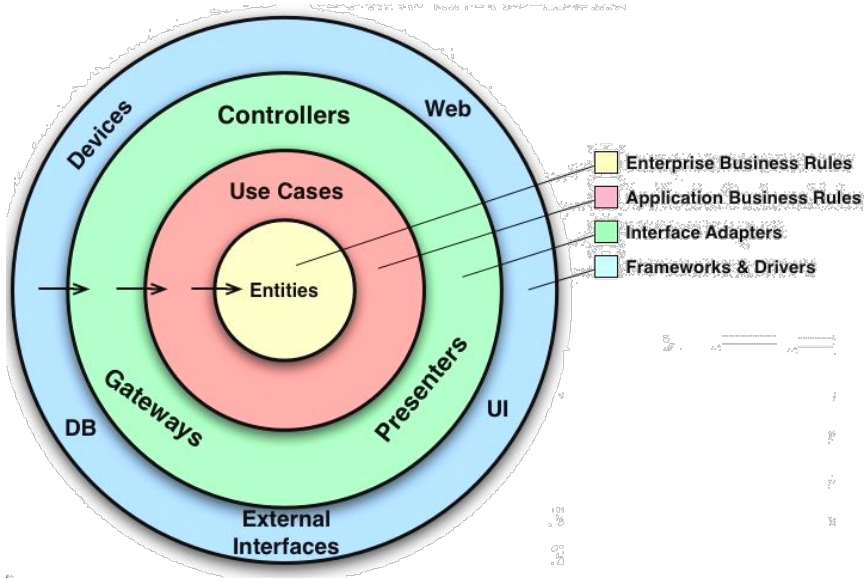
- Developed by Robert C. Martin (Uncle Sam)
- Blog
- Focus on separation of concern and encapsulation
- Universal - can be applied to any software project
- Makes easier swapping components (for example: specific implementation of storage)
- *Nothing in an inner circle can know anything at all about something in an outer circle.*



Clean Architecture: Layers

- Entities: Enterprise wide business rules, usually data models
- Use cases: Application specific business rules, they orchestrate flow of data to and from entities
- Interface adapters (Controllers, Presenters, …): Prepare/convert data for outer layer

Note: 4 layers is just a suggestion, more layers might be necessary for bigger/more complex projects



Dependency Inversion principle

- One of SOLID principles (D)
- ”High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details. Details should depend on abstractions.”
- Main benefit: Components do not depend on implementation specifics but only on defined interfaces that do not have to change when the implementation details change.
- In clean architecture: Inner layers define abstractions, outer layers implement them — so high-level logic depends only on interfaces, never on concrete details.

Dependency Inversion principle

Example: Instead of a Car class directly depending on a GasEngine class, both should depend on an abstract Engine interface. Now the Car can work with any type of engine without directly depending on them.

```
interface Engine {
  fun start()
}

class GasEngine : Engine {
  override fun start() { /* start gas engine */ }
}

class ElectricEngine : Engine {
  override fun start() { /* start electric engine */ }
}

class Car(private val engine: Engine) {
    fun drive() { engine.start() }
}
```

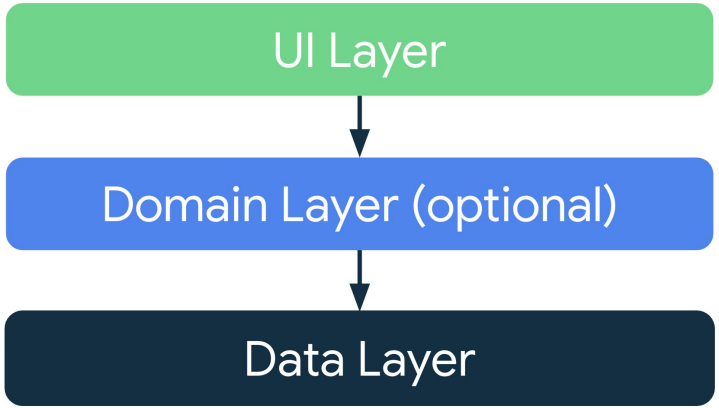
Architecture in Android

Repository Pattern

- The Repository pattern provides a clean API for data access.
- Components:
  - Repository interface: defines API for data access
  - Repository implementation: implements specified API. It is responsible for accessing data from one or more data sources.
- Benefits:
  - Encapsulation of data source and any related logic (caching, processing)
  - Reusability: The interface ensures that the repository can be implemented differently if needed
  - Simplifies testability (easier mocking of repository using interface)

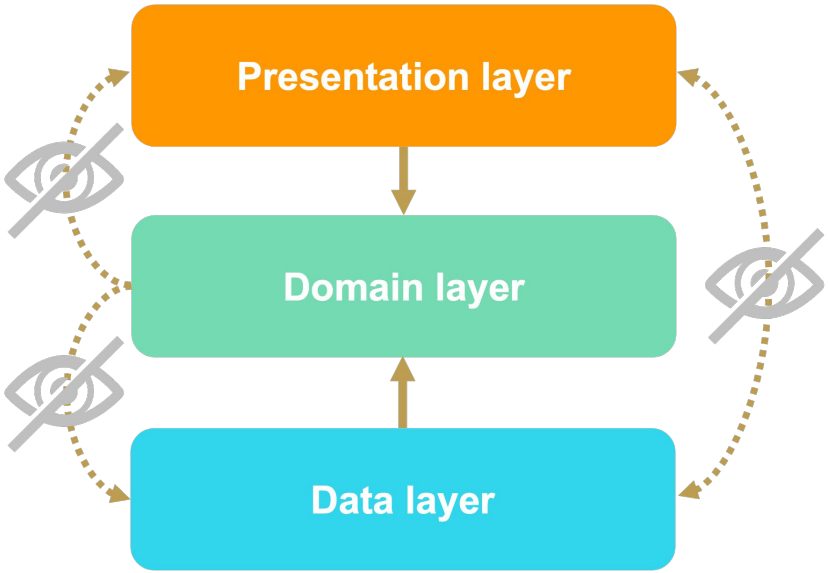
Google Recommended architecture

- Documentation
- UI layer: Displays application data on the screen
- Domain layer: The domain layer is an *optional* layer between the UI and data layers. Responsible for encapsulating complex business logic or simpler business logic that is reused by multiple view models
- Data layer: Contains the business logic of your app and exposes application data



Clean Architecture in Android

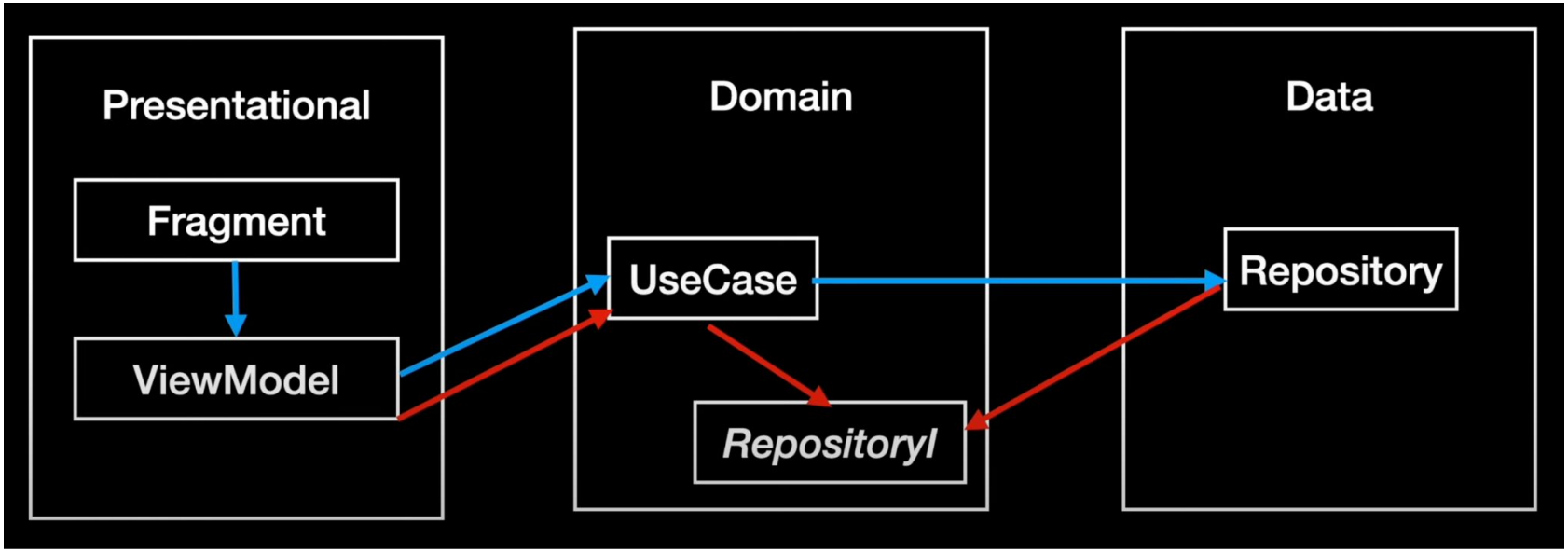
- Three main layers:
  - Data - data storage, network communication, etc.
  - Domain - business logic, data preparation for UI layer
  - UI/Presentation - Presenting data to user
- Data layer depend on Domain layer
- UI/Presentation layer depend on Domain layer
- **Domain layer is independent**



Clean Architecture Components

- **UI**: Activities, Fragments, Composables, ViewModels
- **Domain**:
  - Usecases: Contains one operation, usually calls Repository and/or performs some data processing/formatting. Usecases can be used in multiple ViewModels.
  - Repository interface definition: Definition of how data should be accessed
- **Data**:
  - Repository implementation: Actual implementation of data access

Clean Architecture in Android



Compile time vs. runtime dependencies

Semestral projects: What architecture to use

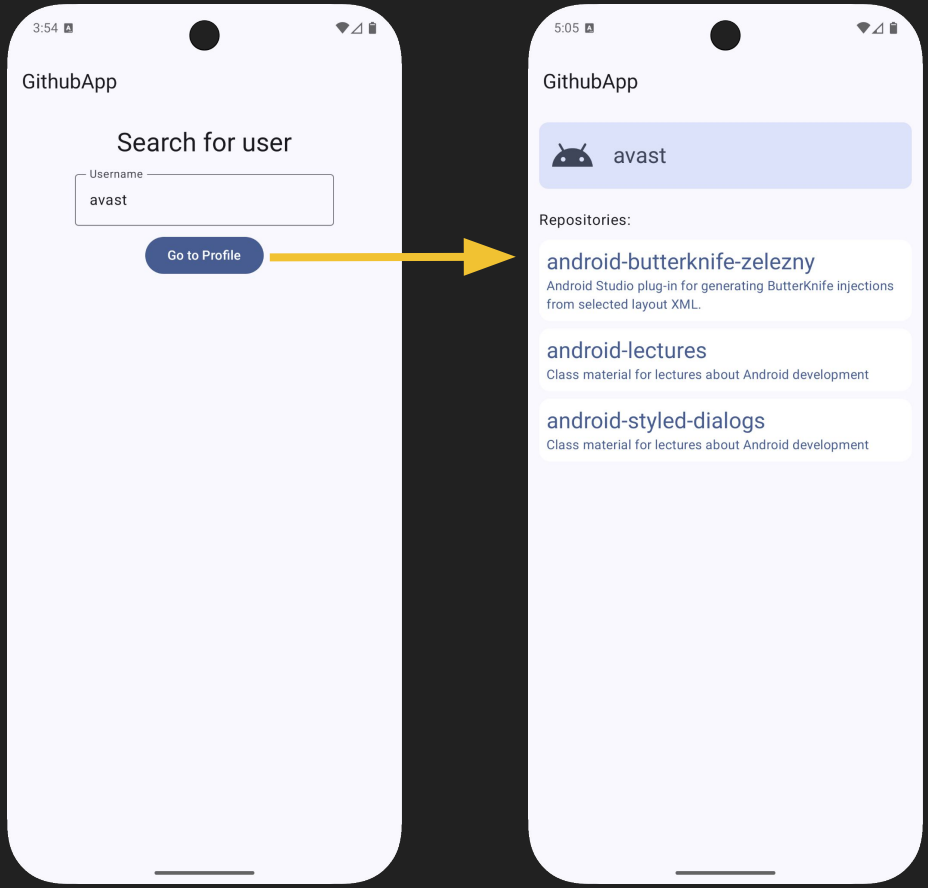
- Separate data and UI layer (architecture recommended by Google)
- Use Repository pattern
- Use MVI or MVVM pattern
- Domain layer or Clean architecture is optional

Github Application:

Create ViewModel for Profile screen that will:

- expose state to UI with Github user information
- use backing property
- contain loadUser(user) method that will update state.
- follow MVVM pattern

Display data provided by VM in Profile screen.


3:54
GithubApp
Search for user
Username
avast
Go to Profile

5:05
GithubApp
avast
Repositories:
android-butterknife-zelezny
Android Studio plug-in for generating ButterKnife injections from selected layout XML.
android-lectures
Class material for lectures about Android development
android-styled-dialogs
Class material for lectures about Android development


DEMO
TIME

Create Retrofit API to access Github API
Use Repository pattern

- getUser
  https://docs.github.com/en/rest/users/users?apiVersion=2022-11-28#get-a-user
- getUserRepositories
  https://docs.github.com/en/rest/repos/repos?apiVersion=2022-11-28#list-repositories-for-a-user

Thank you for attention

Feel free to leave feedback about this lesson: